Beyond the GPU

Why AI is driving a second hardware transition — a systems view for software engineers

Condensed walkthrough • ~1 hour

ROADMAP

Where we are going

1 The driving factors

The physical pressures that push work off the GPU: data movement, memory bandwidth, energy, determinism.

2 The decision framework

Six architectural contracts and a four-condition test for when specialization is actually justified.

3 The families

How beyond-GPU accelerators group by which contract they change — not by vendor.

4 The landscape, grouped

Incumbents, hyperscaler silicon, and challengers — read through the framework, not as a catalog.

5 Economics

Why lifetime cost and software capital decide most outcomes.

6 Conclusions

What holds across the whole compute continuum.

⇒ **Less time on products; more on the physics and economics that make the choice decidable.**

Six hardware ideas, in software terms

GPU

Thousands of parallel lanes. Fast when work is regular, batched, and streamed from high-bandwidth memory.

Matrix / tensor engine

A block that does a whole matrix tile per instruction — amortizes control over many multiply-adds.

HBM

Stacked DRAM giving multi-TB/s. The scarce resource: capacity and bandwidth, not arithmetic.

NPU

An SoC block plus a compiler that maps a whole model graph onto it at low energy.

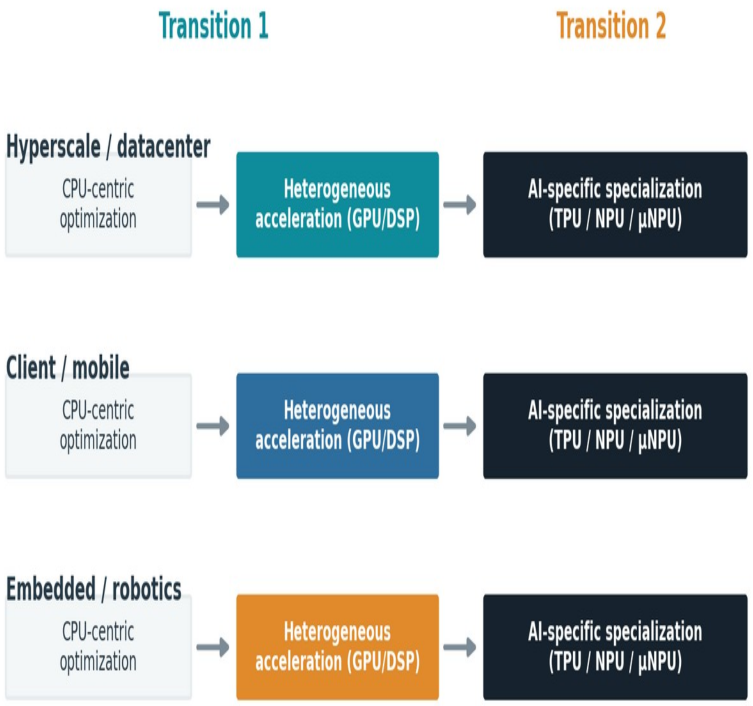
TOPS

A peak arithmetic rating. Rarely the bottleneck — treat as marketing, not a benchmark.

Contract

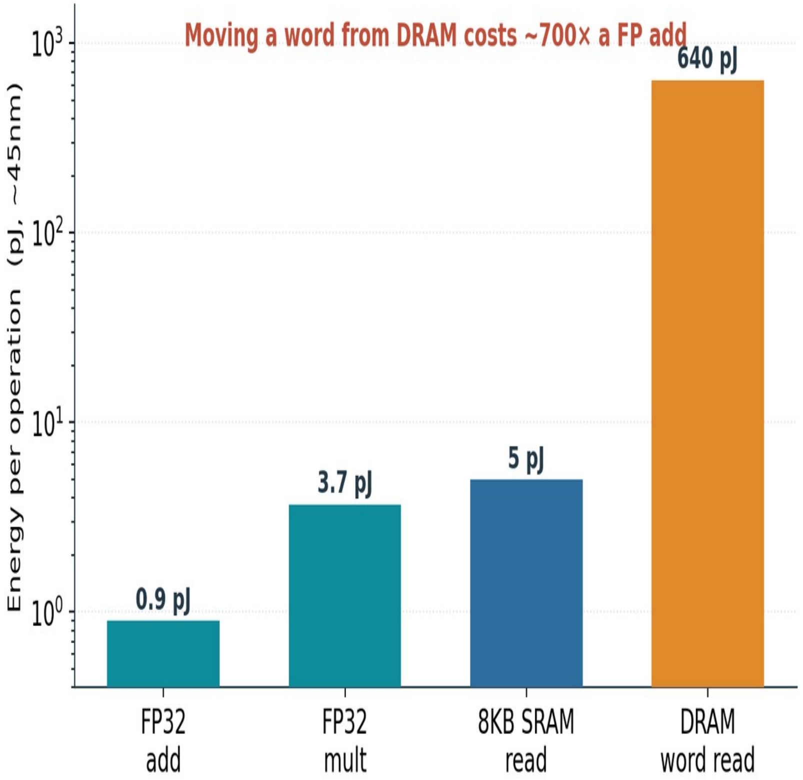
What hardware promises software about execution, memory, numbers, scheduling, comms, and tooling.

Two transitions, across the whole continuum



⇒ **Software optimization keeps hitting limits; each transition adds specialization, it does not replace what came before.**

Data movement — not arithmetic — dominates energy

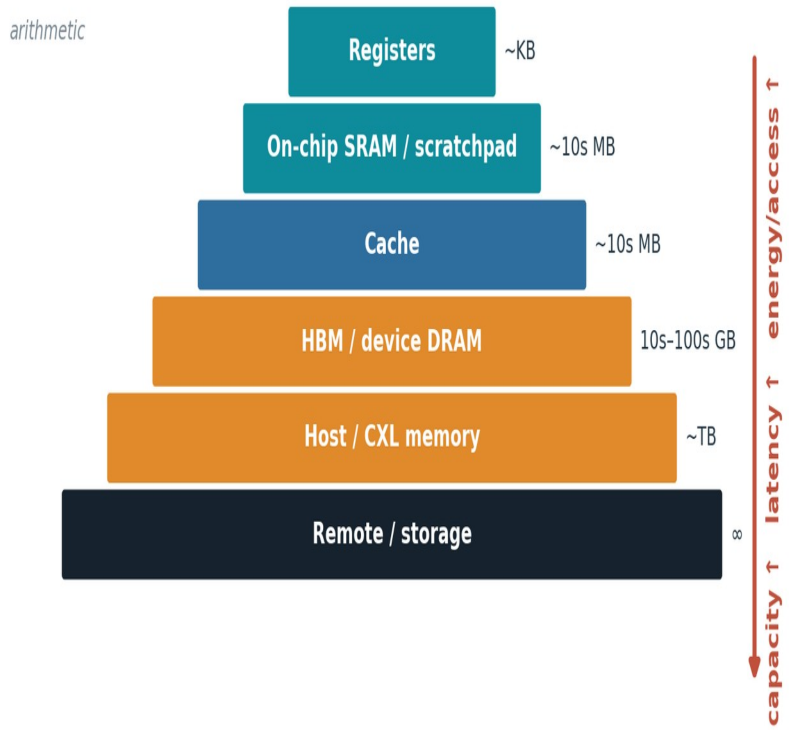


- Compute is nearly free.
- Moving operands is not.
- So locality and reuse set efficiency, not FLOPS.

⇒ **Every architecture that follows is a different answer to 'move less data, or move it a shorter distance.'**

DRIVING FACTOR

Memory is a distance hierarchy

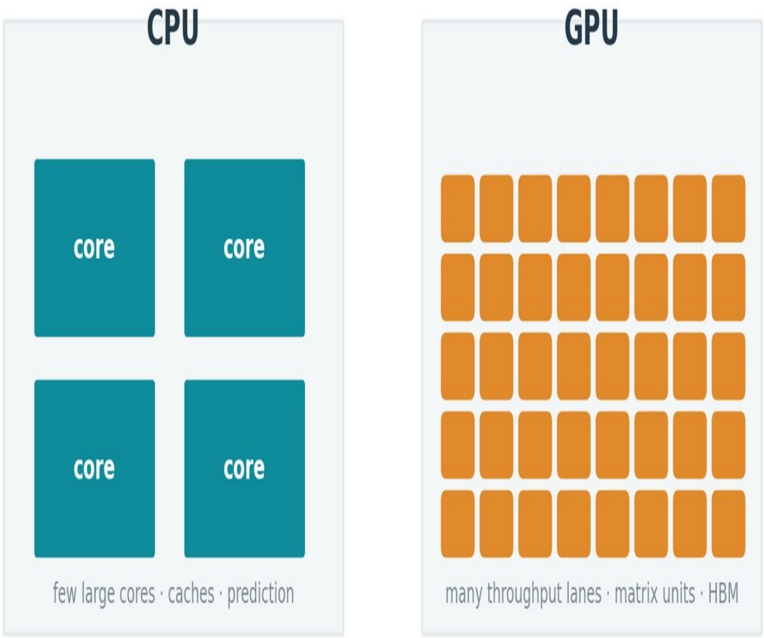


- Each step out: more capacity.
- Also more latency and energy.
- Caches help only if data is reused before eviction.

⇒ **'Memory-bound' is many different problems — each one a lever a specialized design can pull.**

CONCEPT

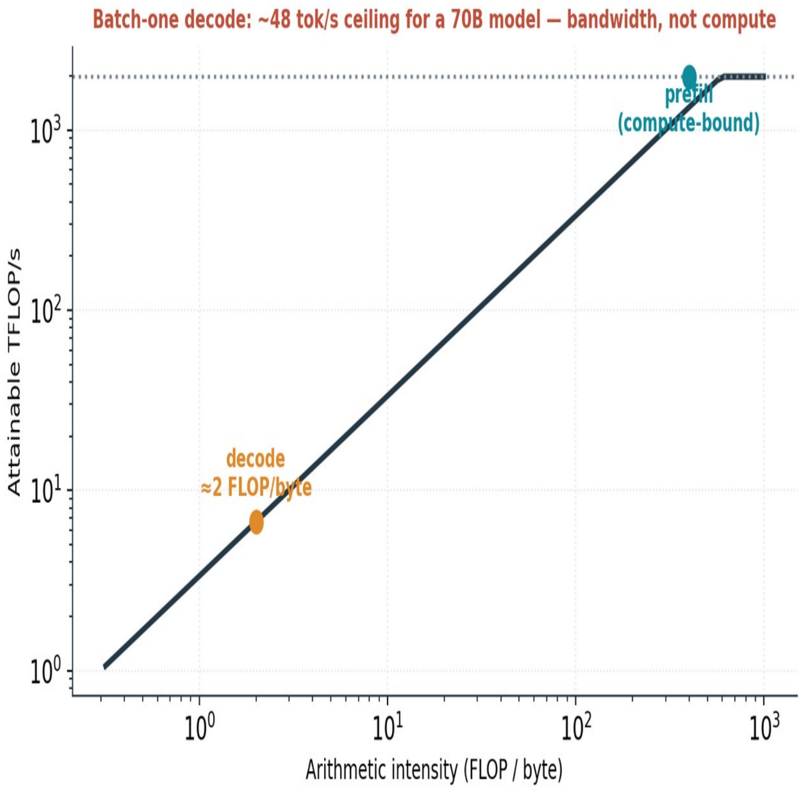
CPU and GPU spend silicon differently



- CPU: low-latency across diverse code.
- GPU: throughput across regular work.
- Neither is optimal for every phase.

⇒ **Specialization narrows what a design does in exchange for more useful work per unit area and energy.**

Batch-one decode is a bandwidth wall

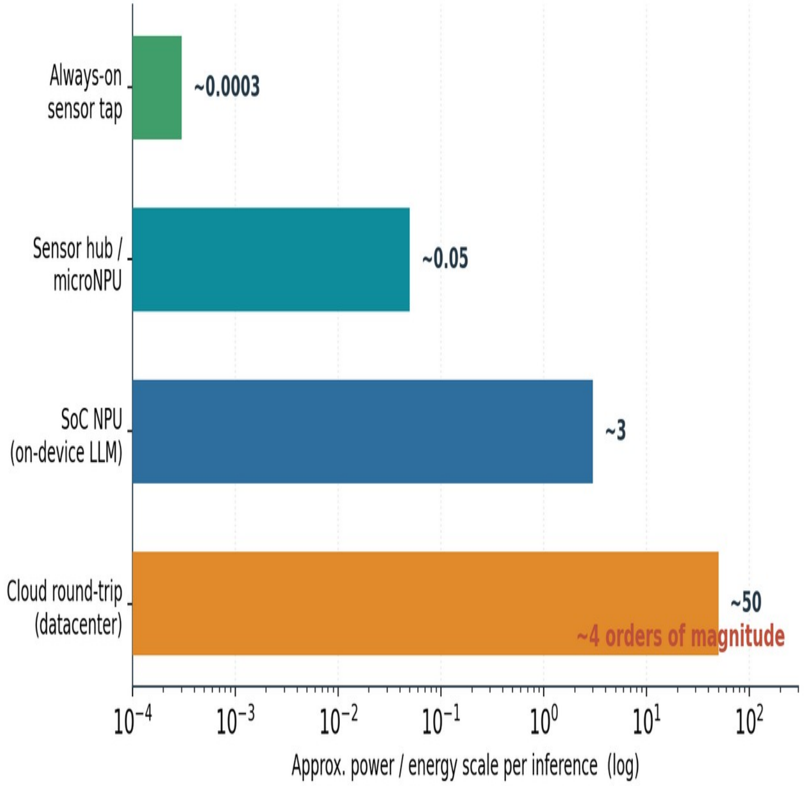


- 70B model, 8-bit, one sequence.
- ≈48 tokens/s ceiling on an H100.
- ~0.3% of the matrix throughput usable.

⇒ **No amount of extra arithmetic moves this — it is set by memory bandwidth. This is the pressure that reshapes serving silicon.**

DRIVING FACTOR

Energy per inference spans four orders of magnitude

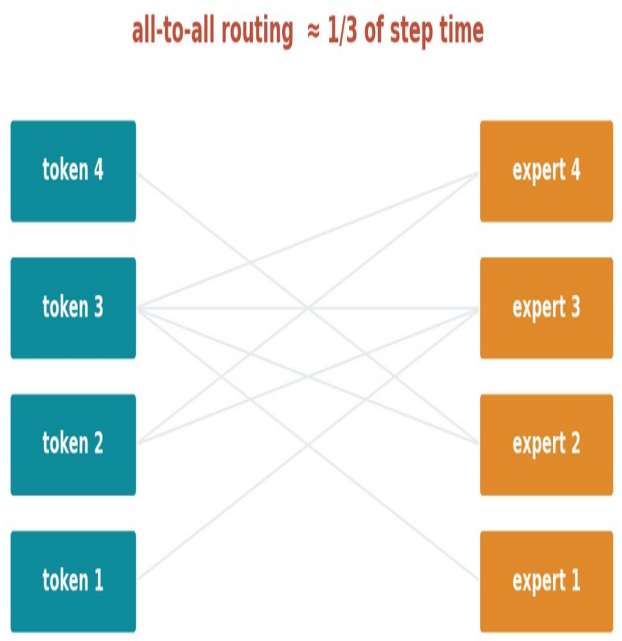


- Always-on tap: microwatts.
- Cloud round-trip: ~watts.
- Which domains wake sets the cost.

⇒ **On-device always-on features are impossible on a GPU contract — the wake-scope bound is structural, not an efficiency gap.**

DRIVING FACTOR

Mixture-of-experts turns compute into communication



⇒ **Routing and all-to-all can be ~1/3 of step time — a bound in the fabric, not the math unit.**

Software succeeds — until it meets physics

3×

IO-aware attention (FlashAttention) throughput

2–4×

KV paging / continuous batching (vLLM)

~34%

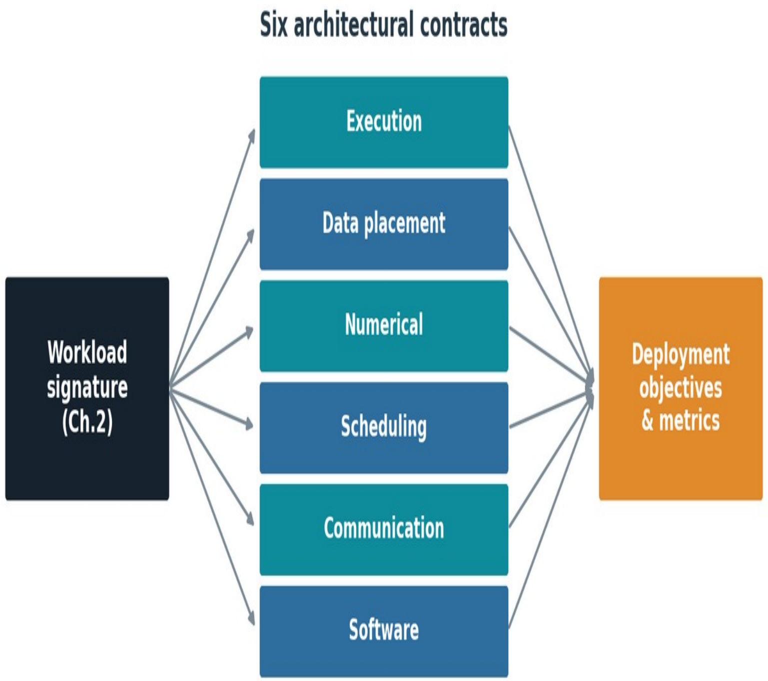
of a training step spent in MoE all-to-all

<300 µW

always-on keyword spotting budget

⇒ **Every gain is real; every gain eventually exposes a residual bound software cannot move. That residue is the case for hardware.**

Hardware defines six contracts



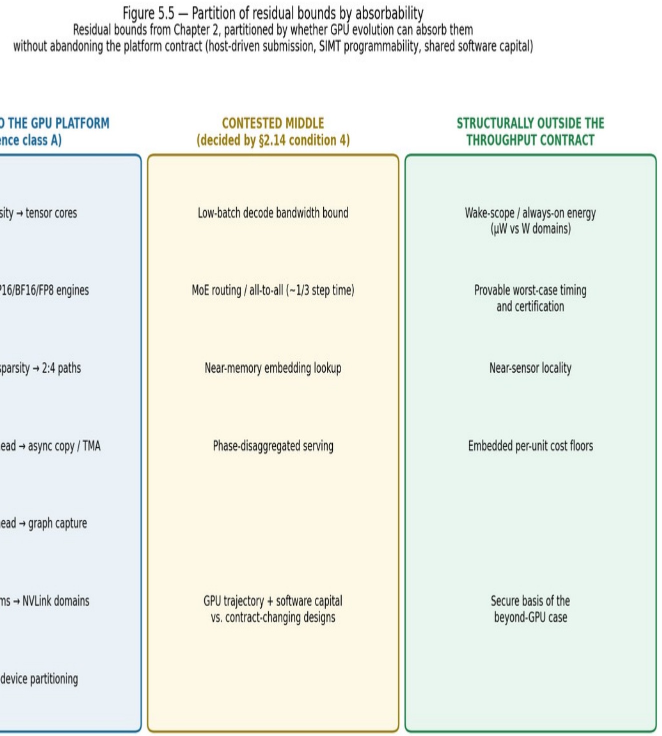
⇒ A workload signature stresses some contracts, not others — that mismatch is where a new architecture earns its place.

Is specialization justified? A four-condition test

- 1 · residual bound remains
- 2 · it has a material consequence
- 3 · hardware changes the bound
- 4 · lifetime economics repay it

⇒ **Same architecture, opposite verdicts. Condition 4 — volume and software cost — decides most cases.**

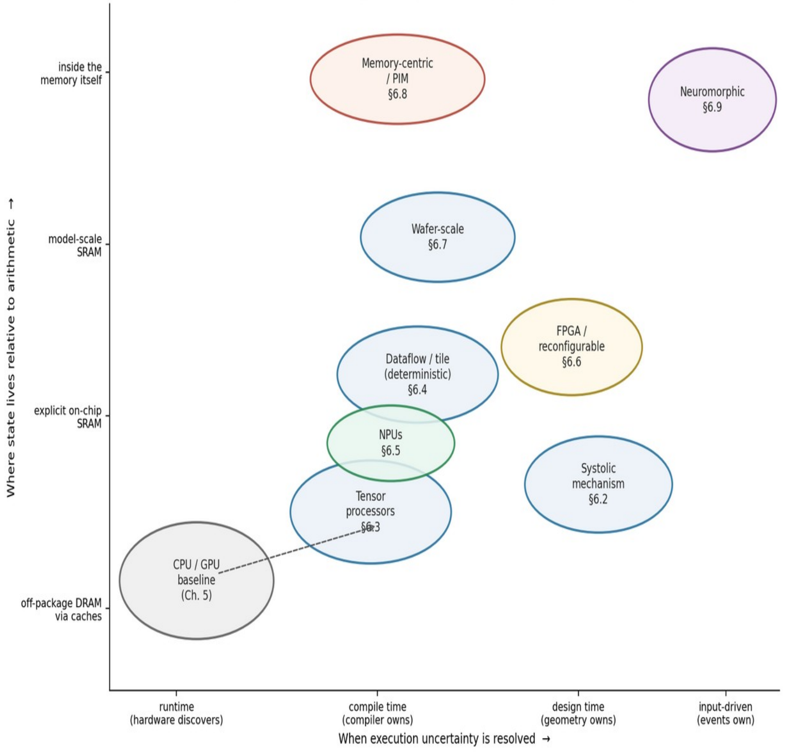
What the GPU absorbs — and what it cannot



⇒ **Absorbable bounds go to the platform; structural bounds (wake scope, determinism, near-sensor, cost floors) belong to specialists.**

One map: when uncertainty resolves × where state lives

Figure 6.1 — The family landscape in contract space (qualitative regions)



⇒ **Every family is a displacement from the CPU/GPU baseline on these two axes — no point dominates.**

THE FAMILIES, GROUPED

Eight families = eight contract changes

Systolic array

Reuse from geometry; data flows through a grid.

Tensor processor

Compiler-owned; matrix engine + managed SRAM (TPU).

Dataflow / tile

Static schedule; deterministic, low-latency (Groq).

NPU / near-sensor

Power-domain-first; wake-scope energy (mobile, µNPU).

FPGA / reconfigurable

Structure set after manufacturing; flexibility.

Wafer-scale

Whole wafer; inter-chip comms become on-wafer.

Memory-centric / PIM

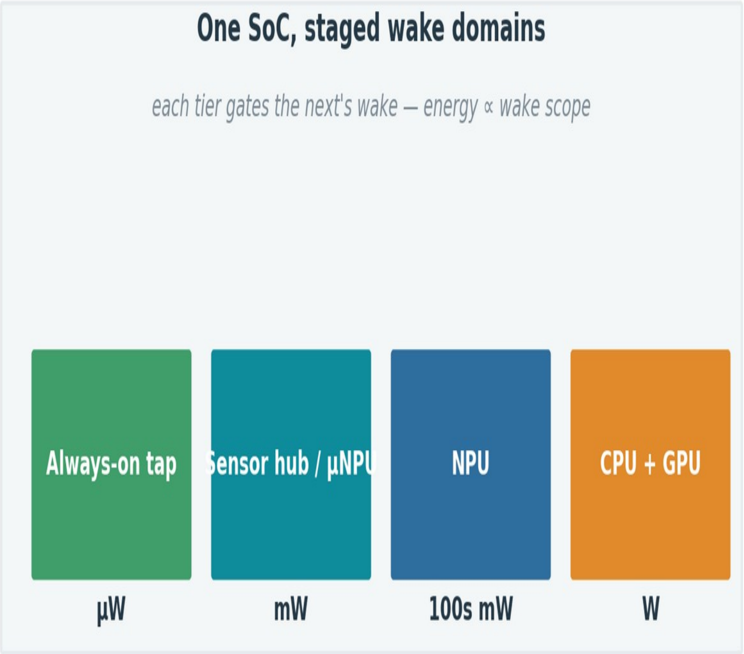
Compute moves to the data; interface carries results.

Neuromorphic

Event-driven; energy ∝ activity (forward-looking).

⇒ **Grouped by contract changed, not by company — the taxonomy outlives the products.**

NPUs own the client and edge

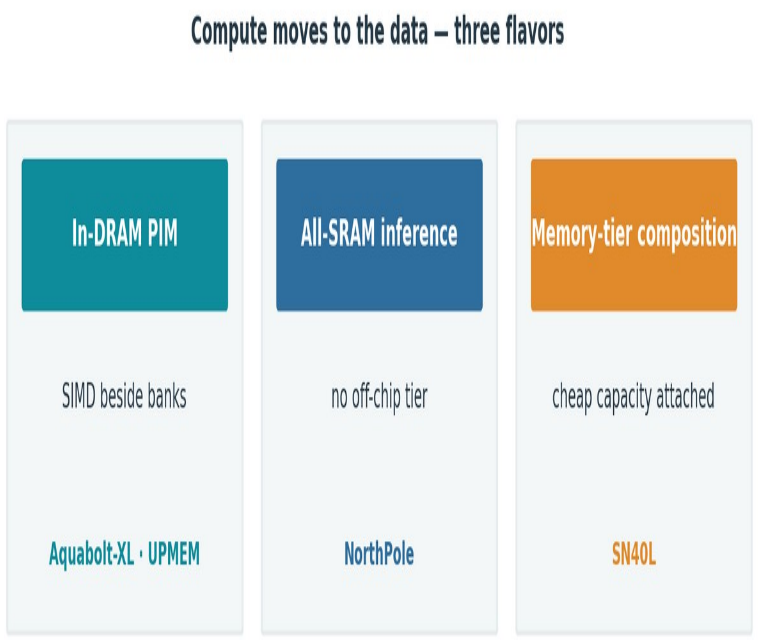


- Value = energy + wake scope.
- Coverage, not TOPS, decides.
- Fallback behavior is everything.

⇒ **Wake-scope energy and per-unit cost floors are structurally outside the GPU contract.**

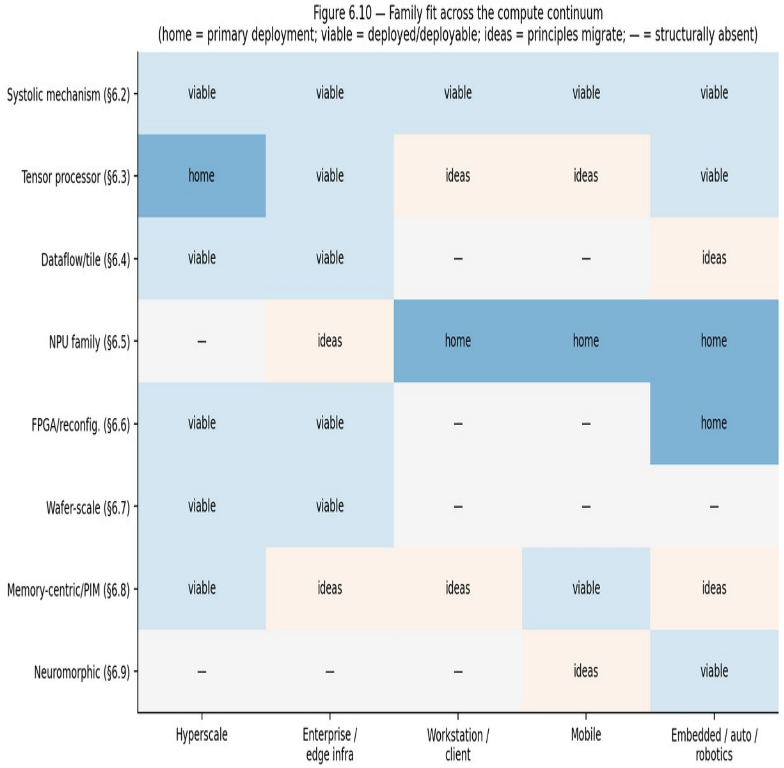
STRONGEST LEVER

Memory-centric designs attack the measured bounds



⇒ **Best condition-3 case in the survey — gated only by the lack of a portable programming model.**

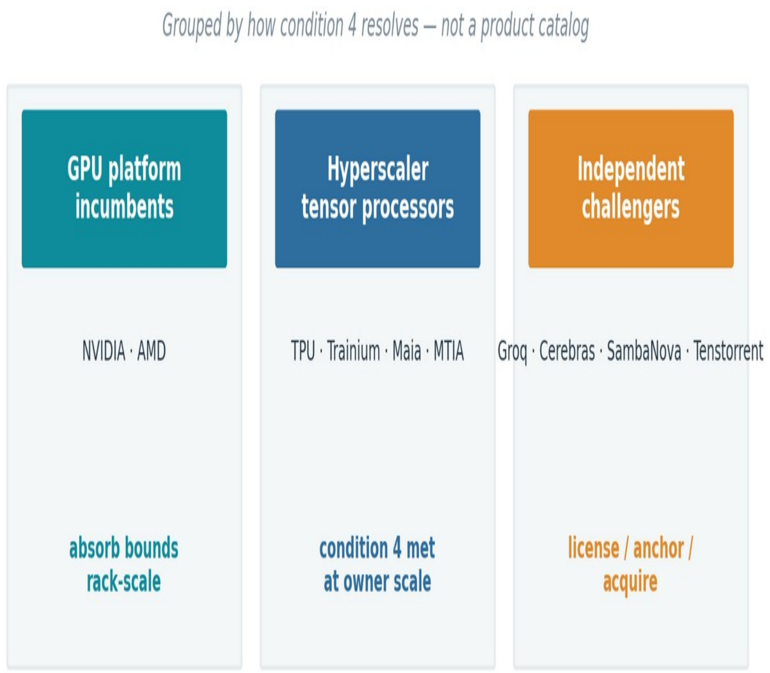
Same families, different binding constraints



⇒ **Ideas migrate both ways; implementations diverge because each deployment prices the stressors differently.**

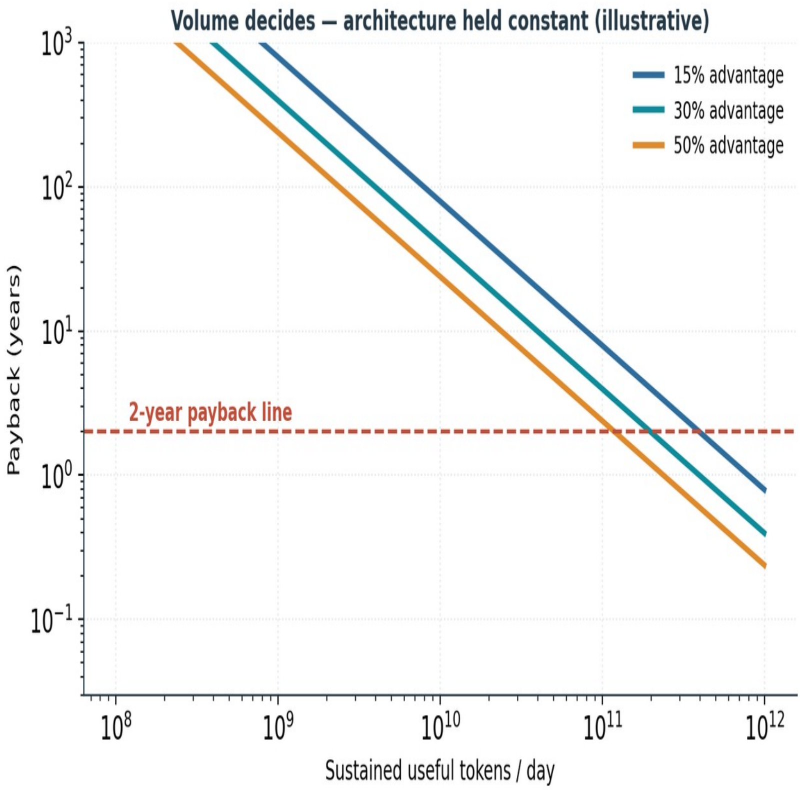
THE LANDSCAPE, GROUPED

Three groups, one deciding variable



⇒ **None of the challengers failed on architecture — every trajectory was decided by condition 4.**

Volume decides — architecture held constant



- Fixed cost ≈ software enablement.
- Recurring savings scale with volume.
- Flexibility has option value.

⇒ **Below a volume threshold, no efficiency percentage rescues the case; above it, small percentages justify large programs.**

CONCLUSIONS

What holds across the whole picture

A **Software converges on physics**

Both transitions are software succeeding until it exposes a hard, measurable bound.

B **Necessity is decidable**

A four-condition test returns specialize / market-no / stay-on-GPU — from measurable inputs.

C **Condition 4 dominates**

Software capital, volume, and distribution decide outcomes more than silicon merit.

D **Families, not a successor**

Each changes a contract the GPU cannot adopt without ceasing to be a GPU.

E **Bounds moved to memory & power**

Capacity, bandwidth, fabric, and facility power now separate contenders — not FLOPS.

F **One continuum**

The same taxonomy reads microwatt silicon and megawatt pods.

THE ARGUMENT IN ONE SENTENCE

Software optimization succeeds until it exposes a bound; the bound is absorbed by the GPU platform, escaped by changing a contract, or lived with — and the deployment class decides which.

Everything else — signatures, contracts, families, economics — is the machinery that makes that choice decidable.